

Faculdade Unialfa

Caio Fábio

Sistema de Extração de dados e sites do mercado financeiro

Goiânia
2026
Caio Fábio

Sistema de Extração de dados e sites do mercado financeiro

Relatório final, apresentado a
Universidade _____, como parte das
exigências para a obtenção do título de
_____.

Local, ____ de _____ de ____.

BANCA EXAMINADORA

Prof. (Nome do orientador)
Afiliações

Prof. (Nome do professor avaliador)
Afiliações

Prof. (Nome do professor avaliador)
Afiliações

- **1 INTRODUÇÃO**

- 1.1 TEMA E CONCEITO DA IDEIA
- 1.2 PROBLEMATIZAÇÃO
- 1.3 SOLUÇÃO PROPOSTA

- **2 ESTUDO DE VIABILIDADE**

- 2.1 VIABILIDADE TÉCNICA
- 2.2 VIABILIDADE ECONÔMICA
- 2.3 VIABILIDADE OPERACIONAL

- **3 DESENVOLVIMENTO DO SISTEMA**

- 3.1 DEFINIÇÕES E CONCEITOS CHAVE
- 3.2 DESCRIÇÃO DA ARQUITETURA E TECNOLOGIAS
- 3.3 ENGENHARIA DE REQUISITOS
 - 3.3.1 Requisitos Funcionais (RF)
 - 3.3.2 Requisitos Não-Funcionais (RNF)

- **4 MODELAGEM E DIAGRAMAS UML**

- 4.1 DIAGRAMA DE CASOS DE USO
- 4.2 DIAGRAMA DE ARQUITETURA E FLUXO DE DADOS

- **5 PROTOTIPAGEM DE INTERFACES (TELAS)**

- 5.1 DASHBOARD DE MONITORAMENTO OPERACIONAL

- **REFERÊNCIAS**

INTRODUÇÃO

1.1 Tema e Conceito da Ideia

O projeto **FinExtract** consiste no desenvolvimento de uma infraestrutura robusta de *Data Engineering* voltada ao mercado financeiro. O conceito central baseia-se em uma plataforma de arquitetura orientada a eventos (*Event-Driven Architecture*) dedicada à raspagem, higienização, estruturação e distribuição de dados financeiros públicos e privados de forma assíncrona e resiliente.

1.2 Problematização

A análise do mercado financeiro sofre com três gargalos informacionais:

1. **Fratura Estrutural (Quebra de Código):** Scripts comuns de raspagem falham quando portais alteram suas tags HTML, gerando "silêncio de dados".
2. **Bloqueios de Infraestrutura:** Sistemas de segurança (WAF como Cloudflare) barram robôs por meio de CAPTCHAs e bloqueios de IP.
3. **Inconsistência Analítica:** Dados coletados manualmente ou de fontes heterogêneas apresentam divergências de formato (ex: strings monetárias, fuso horário), impossibilitando o uso imediato em algoritmos preditivos.

1.3 Solução Proposta

O FinExtract resolve essas dores isolando o processo de extração em microsserviços integrados por mensageria (Apache Kafka). Ele utiliza técnicas de *Stealth Scraping* para mitigar bloqueios, aplica um motor rigoroso de ETL para padronização matemática e disponibiliza uma API RESTful de baixa latência com cache em memória (Redis).

ESTUDO DE VIABILIDADE

2.1 Viabilidade Técnica

O projeto demonstra-se altamente viável. A stack tecnológica escolhida (Python, Docker, Kafka, PostgreSQL e Redis) é de código aberto (*open-source*), possui ampla maturidade no mercado e vasta documentação. A equipe de desenvolvimento possui competência nas bibliotecas de automação de navegadores (Playwright) e modelagem de banco de dados relacionais e séries temporais.

2.2 Viabilidade Econômica

A implementação de uma solução *in-house* elimina a dependência de licenças corporativas proibitivas (como terminais Bloomberg ou Economatica, que custam milhares de dólares anuais por usuário). O custo de infraestrutura em nuvem (AWS ou Google Cloud) para processar o volume planejado é marginal quando comparado ao ganho financeiro obtido pela velocidade de acesso às informações estratégicas.

2.3 Viabilidade Operacional

O sistema operará de forma autônoma (24/7), exigindo intervenção humana apenas para manutenção corretiva quando uma fonte de dados mudar drasticamente seu ecossistema. A interface de monitoramento integrada permitirá que a equipe de TI ou de operações financeiras gerencie a saúde dos robôs sem a necessidade de manipular linhas de código.

DESENVOLVIMENTO DO SISTEMA

3.1 Definições e Conceitos Chave

- **Web Scraping / Crawling:** Técnica de computação que transforma dados não estruturados da web (HTML) em dados estruturados.
- **ETL (Extract, Transform, Load):** Processo de extração de dados, transformação para limpeza e padronização, e carregamento em um repositório final.
- **Idempotência:** Propriedade que garante que uma operação executada múltiplas vezes resulte no mesmo estado, impedindo a duplicação de registros financeiros.

3.2 Descrição da Arquitetura e Tecnologias

- **Python (Playwright & Scrappy):** Linguagem core e frameworks utilizados para emulação de navegação humana e extração rápida.
- **Apache Kafka:** Cluster de mensageria responsável por receber os dados brutos e garantir que nenhuma mensagem seja perdida caso o banco de dados sofra oscilações.
- **PostgreSQL (com extensão TimescaleDB):** Banco de dados relacional otimizado para séries temporais e dados estruturados complexos.
- **Redis:** Banco em memória utilizado como camada de cache para entregar as últimas cotações à API com latência inferior a 10 milissegundos.
- **Docker / Kubernetes:** Tecnologias de containerização para garantir que o sistema escale horizontalmente adicionando mais robôs conforme a demanda.

3.3 Engenharia de Requisitos

3.3.1 Requisitos Funcionais (RF)

- **RF-001:** O sistema deve extrair dados de balanços patrimoniais diretamente do portal da CVM de forma diária.

- **RF-002:** O sistema deve extrair cotações de ativos da B3 em intervalos de 1 minuto durante o horário do pregão.
- **RF-003:** O sistema deve padronizar todas as strings monetárias capturadas em formato `Decimal` numérico.
- **RF-004:** O sistema deve disponibilizar uma API endpoints para consulta de dados históricos e atuais.
- **RF-005:** O sistema deve disparar alertas visuais e via mensageria caso um robô de extração seja bloqueado por um site alvo.

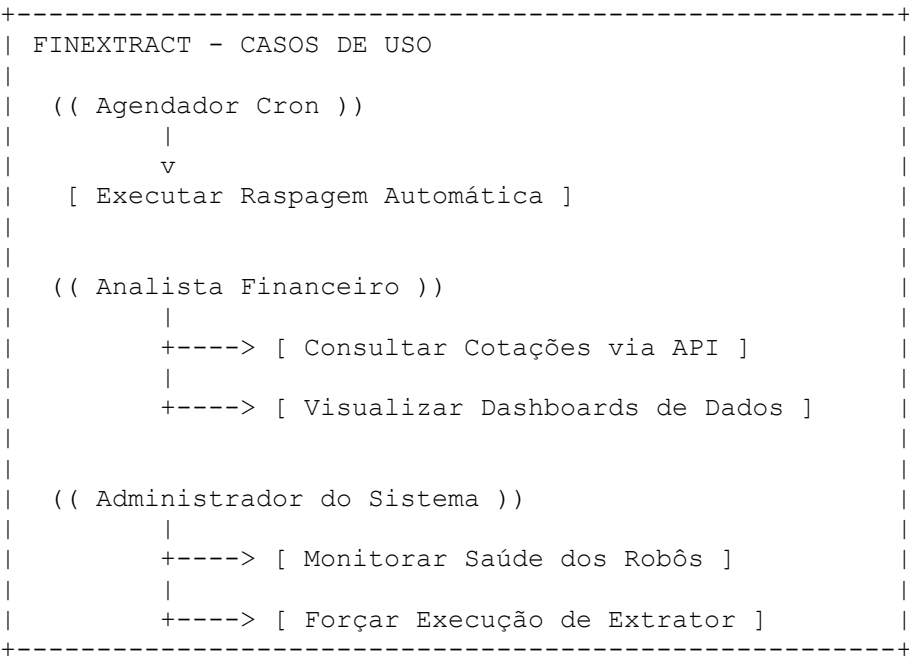
3.3.2 Requisitos Não-Funcionais (RNF)

- **RNF-001 (Desempenho):** A API de consumo de dados deve responder às requisições em tempo inferior a 50ms sob carga normal.
- **RNF-002 (Disponibilidade):** O ecossistema de ingestão deve possuir disponibilidade de 99.5% (Uptime).
- **RNF-003 (Segurança):** Todas as credenciais de APIs externas e chaves de banco de dados devem ser criptografadas e armazenadas em um cofre de senhas (Vault).
- **RNF-004 (Resiliência):** O sistema deve rotacionar IPs de proxy a cada requisição mal sucedida para evitar detecção.

MODELAGEM E DIAGRAMAS UML

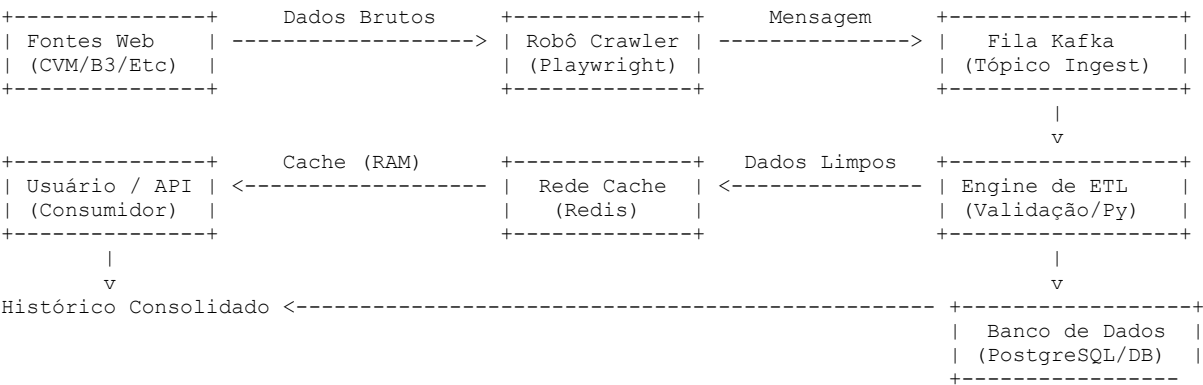
4.1 Diagrama de Casos de Uso

O diagrama abaixo mapeia as interações dos atores (Analista Financeiro, Administrador e o próprio Sistema Automatizado) com as fronteiras do software.



4.2 Diagrama de Arquitetura e Fluxo de Dados

Abaixo está representado o fluxo de ponta a ponta do pipeline de engenharia de dados.



PROTOTIPAGEM DE INTERFACES (TELAS)

5.1 Dashboard de Monitoramento Operacional

Interface administrativa em modo texto/wireframe representando a console de gerenciamento do sistema (*FinExtract Ops Cockpit*).

```
=====
FINEXTRACT v1.0 // OPERATIONAL COCKPIT [ STATUS: ESTÁVEL ]
=====

[MÓDULOS DE INGESTÃO]
-> Extrator_CVM_Balanços : [ ATIVO ] | Última execução: Há 12min (Sucesso)
-> Extrator_B3_Tickers : [ ATIVO ] | Taxa de Atualização: 60s
-> Extrator_Anbima_Taxas : [ ALERTA ] | Erro: Mudança de seletor HTML (HTTP
200)

[MÉTRICAS DO PIPELINE]
+-----+-----+-----+
| Requisições / Min | Registros Processados | Latência Média API |
| 1.420 req/m | 8.429.110 linhas | 12 ms |
+-----+-----+-----+

[FILA DE MENSAGENS (KAFKA)]
Tópico: cvm.raw.events | Lag: 0 mensagens (Em dia)
Tópico: b3.quotes.raw | Lag: 4 mensagens (Processando rápido)

[AÇÕES DO SISTEMA]
[1] Forçar Reinicialização de Robô [2] Pausar Ingestão B3 [3] Ver Logs Erro
=====
```

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 15287**: Informação e documentação - Projeto de pesquisa - Apresentação. Rio de Janeiro: ABNT, 2011.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML: Guia do Usuário**. 2. ed. Rio de Janeiro: Elsevier, 2012.

KLEPPMANN, Martin. **Designing Data-Intensive Applications**: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol: O'Reilly Media, 2017.